

Summer Bootcamp – Session 3

Portfolio Optimization Project

Nathan Sauldubois

August 2026

Optional project

This optional extension can be completed independently by students who want to explore portfolio optimization further. The aim is to build a complete and reproducible research pipeline: simulate train and test returns, study the covariance structure with PCA, estimate several portfolios, and evaluate rebalanced strategies without look-ahead bias. The central question is:

Does a better-conditioned covariance estimate produce more stable weights and better realized risk after transaction costs?

The stages and self-checks provide a suggested path that students can follow at their own pace. They may stop after any stage or select the parts that interest them. A partial pipeline with correct validation is more useful than a more sophisticated strategy containing look-ahead bias.

Files supplied and suggested outputs

The folder contains this statement and `portfolio_project_skeleton.py`. A complete implementation can create `returns_train.csv`, `returns_test.csv`, and `true_parameters.npz`. Store tables and figures in clearly named output folders. Do not edit the generated CSV files manually.

1 Definitions and conventions

Returns, moments, and annualization

For n assets, $R_t = (R_{1,t}, \dots, R_{n,t})^\top$ is the random vector of daily simple returns. A sample contains rows indexed by date and one column per asset. The population daily moments are

$$\mu_d = \mathbb{E}[R_t], \quad \Sigma_d = \text{Cov}(R_t).$$

Under the usual independent-return annualization convention,

$$\mu = 252\mu_d, \quad \Sigma = 252\Sigma_d.$$

This convention is an approximation for arithmetic returns; state it in the report rather than treating it as an identity for compounded wealth.

Portfolio return, risk, and constraints

A portfolio is a vector $w \in \mathbb{R}^n$. Full investment means $\mathbf{1}^\top w = 1$; long-only means $w_i \geq 0$. Its expected return and variance are

$$R(w) = \mu^\top w, \quad \sigma^2(w) = w^\top \Sigma w.$$

Equal weight uses $w_i = 1/n$. The Herfindahl concentration index is $H(w) = \sum_i w_i^2$; under long-only full investment it ranges from $1/n$ to 1.

Portfolio optimization problems

The global minimum-variance portfolio (GMV) solves

$$\min_w \frac{1}{2} w^\top \Sigma w \quad \text{s.t.} \quad \mathbf{1}^\top w = 1$$

with any additional bounds stated in the question. A mean–variance portfolio solves

$$\min_w \frac{1}{2} w^\top \Sigma w - \gamma \mu^\top w, \quad \gamma \geq 0.$$

For annual risk-free rate r_f , the Sharpe ratio is

$$\text{SR}(w) = \frac{\mu^\top w - r_f}{\sqrt{w^\top \Sigma w}}.$$

A maximum-Sharpe portfolio maximizes this ratio over its feasible set. If the denominator is numerically zero, the candidate is invalid.

Covariance estimators used in the project

Let $S = U \text{diag}(\lambda_1, \dots, \lambda_n) U^\top$, with decreasing eigenvalues. The rank- k PCA reconstruction is

$$S_{\text{PCA}}^{(k)} = U_k \text{diag}(\lambda_1, \dots, \lambda_k) U_k^\top.$$

Use the diagonal-corrected version

$$\tilde{S}_{\text{PCA}}^{(k)} = S_{\text{PCA}}^{(k)} + \text{diag}\left(\text{diag}(S - S_{\text{PCA}}^{(k)})\right)$$

for optimization. Ridge regularization uses

$$S_\delta = S + \delta I, \quad \delta = \tau \text{tr}(S)/n.$$

The scale-free parameter τ makes experiments easier to compare.

2 Suggested Python architecture

Use the supplied skeleton and implement the functions below in order. Keep the function names and return types so that later stages can reuse earlier work. Each function should validate its own output before returning it.

- A.1. Data generation.** Implement `factor_covariance`, `generate_population_parameters`, `theoretical_moments`, `simulate_returns`, and `generate_and_save_data`. The final function should return two data frames and create the two CSV files and parameter archive. Add assertions for shapes, dates, column names, finite values, and covariance eigenvalues.
- A.2. Loading and descriptive checks.** Implement `load_project_data` and `annualized_sample_moments`. The loader should parse dates, reject overlapping train/test indices, preserve asset order, and compare stored dimensions with the configuration.
- A.3. PCA.** Implement `fit_pca_on_train`, `pca_covariance`, `pca_scores`, `principal_angles`, and `plot_pca_diagnostics`. Return eigenvalues in decreasing order and use the training mean and loadings when transforming test observations.
- A.4. Portfolio construction.** Implement `ridge_covariance`, `unconstrained_gmv`, `solve_long_only_gmv`, `solve_max_sharpe`, `solve_mean_variance`, and `trace_efficient_frontier`. Every numerical solver

should return both weights and its raw solver result. Never silently accept a failed solver.

- A.5. Optimality and feasibility checks.** Implement `portfolio_diagnostics` and `directional_optimality_check`. At minimum report budget error, bound violations, predicted variance, concentration, solver status, and whether small feasible perturbations reduce the objective.
- A.6. Model selection.** Implement `select_covariance_hyperparameters`. Its output should contain one row per candidate, the exact estimation and validation dates, realized validation risk, concentration, and the declared selection criterion. The function should not accept test data as an argument.
- A.7. Backtesting.** Implement `drift_weights`, `frozen_weight_backtest`, `backtest_rebalanced_strategy`, and `performance_metrics`. Return aligned series or data frames for target weights, pre-trade weights, gross returns, net returns, turnover, costs, wealth, and drawdown.
- A.8. Reporting and robustness.** Implement `plot_backtest_diagnostics` and `run_one_seed`. The top-level main function can call the stages in order and regenerate all files from an empty output folder without manual intervention.

Recommended coding structure

Avoid putting the full project in one notebook cell or one function. Each optimizer, estimator, validation rule, metric, and plot should be callable and testable independently. Use informative exceptions for invalid inputs and store solver diagnostics alongside portfolio weights.

3 Detailed function contracts

The following contracts describe a complete implementation. Shapes use T for the number of observations, n for assets, k for retained components, and m for equality constraints. Unless specified otherwise, numeric arrays must be finite and use floating-point values.

Utilities

`make_output_directories(cfg)` **Input:** one `Config`. **Work:** create the data, figure, and table directories with `parents=True` and `exist_ok=True`; never delete existing results. **Return:** `None`. **Check:** every expected path exists and is a directory.

`asset_names(n_assets)` **Input:** a positive integer. **Work:** generate zero-padded names from `Asset_01` through `Asset_NN`. **Return:** an ordered list of unique strings of length n . **Reject:** nonpositive or noninteger input.

`check_covariance(sigma, name, tol)` **Input:** a candidate matrix, a diagnostic name, and tolerance. **Work:** verify square shape, finite entries, symmetry, and $\lambda_{\min} \geq -\text{tol}$. **Return:** `None` on success. **Failure:** raise an informative `ValueError` containing `name`, the failed property, and the observed value.

Stage 1: simulation and files

`factor_covariance()` **Input:** none. **Work:** create μ_f , factor volatilities, and R_f , then compute $\Sigma_f = \text{diag}(s_f)R_f\text{diag}(s_f)$ in annual units. **Return:** three arrays with shapes $(3,)$, $(3,)$, and $(3,3)$. **Checks:** correlation diagonal equals one, covariance diagonal equals s_f^2 , and covariance is positive semidefinite.

`generate_population_parameters(rng, cfg)` **Inputs:** a NumPy generator and configuration. **Work:** draw α , B , and idiosyncratic volatilities with the stated distributions; build D ; attach factor moments. **Return:** a dictionary containing `alpha`, `B`, `D`, `idio_vol`, `mu_f`, `factor_vol`, and `sigma_f`. **Checks:** exact shapes, positive volatilities, diagonal D , and finite values.

`theoretical_moments(params)` **Input:** the population dictionary. **Work:** compute annual $\mu_{\text{true}} = \alpha + B\mu_f$ and $\Sigma_{\text{true}} = B\Sigma_f B^\top + D$. **Return:** arrays of shapes $(n,)$ and (n,n) . **Checks:** expected keys and dimensions, symmetry, and positive semidefiniteness.

`simulate_returns(rng, n_obs, params, cfg)` **Inputs:** generator, positive observation count, population parameters, and configuration. **Work:** draw factor and idiosyncratic shocks in daily units and combine them as $\alpha/252 + f_t B^\top + \varepsilon_t$. **Return:** a (T, n) array of daily simple returns. **Checks:** exact shape, finite values, and no mutation of `params`.

`returns_frame(values, start_date, cfg)` **Inputs:** a (T, n) array and valid start date. **Work:** attach an increasing business-day index and standard asset names. **Return:** a data frame with named date index. **Checks:** n columns, unique dates, no missing values, and column order equal to `asset_names(n)`.

`generate_and_save_data(cfg)` **Input:** configuration. **Work:** spawn independent RNG streams, draw parameters once, simulate independent train and test shocks, create non-overlapping dates, compute true moments, and save both CSV files plus the NPZ archive. **Return:** train and test data frames. **Checks:** reload the saved files, compare shapes and column order, and verify that a second run with the same seed reproduces values exactly.

`load_project_data(cfg)` **Input:** configuration containing file paths. **Work:** read CSV dates, load the NPZ archive, and reconstruct the parameter dictionary. **Return:** train data frame, test data frame, and parameter dictionary. **Reject:** missing files, wrong dimensions, overlapping dates, reordered assets, nonnumeric returns, or missing parameter keys.

`annualized_sample_moments(returns, cfg)` **Input:** one return data frame in daily units. **Work:** compute $252\bar{r}$ and 252 times the sample covariance with the usual $T - 1$ denominator. **Return:** $(n,)$ mean and (n, n) covariance. **Checks:** at least two observations, all-numeric columns, finite moments, and covariance validity.

Stage 2: PCA

`fit_pca_on_train(train)` **Input:** training returns only. **Work:** store the training mean, center the data, compute daily sample covariance, apply `numpy.linalg.eigh`, and sort eigenpairs decreasingly. **Return:** a dictionary with `mean`, `eigenvalues`, `eigenvectors`, `explained_ratio`, and `empirical_covariance`. **Checks:** centered means near zero, orthonormal eigenvectors, nonincreasing eigenvalues, explained ratios summing to one, and full reconstruction error below tolerance.

`pca_covariance(eigenvalues, eigenvectors, n_components, empirical_covariance, diagonal_correction)` **Inputs:** fitted eigenpairs, integer k , empirical covariance, and correction flag. **Work:** reconstruct the rank- k covariance and, when requested, add the diagonal residual. **Return:** one symmetric (n, n) covariance in the same units as the input. **Checks:** $1 \leq k \leq n$, compatible shapes, diagonal preservation when corrected, and positive semidefiniteness.

`pca_scores(returns, training_mean, training_eigenvectors, n_components)` **Inputs:** train or test returns and PCA quantities fitted on train. **Work:** subtract the training mean and multiply by U_k ; never refit or recenter with the test mean. **Return:** a (T, k) data frame named `PC_01, \dots`, with the original dates. **Checks:** matching asset dimension and valid k .

`principal_angles(empirical_vectors, theoretical_vectors, n_components)` **Inputs:** two orthonormal bases and k . **Work:** compute singular values of $U_{\text{true},k}^\top U_{\text{emp},k}$, clip to $[-1, 1]$, and take arccos. **Return:** k angles in radians, preferably sorted increasingly. **Checks:** orthonormality and angles in $[0, \pi/2]$ up to tolerance.

`plot_pca_diagnostics(pca, output_path)` **Inputs:** PCA dictionary and PNG path. **Work:** make aligned scree and cumulative-explained-variance panels, mark $k = 3$, label units and axes, and call `tight_layout`. **Return:** `None`. **Checks:** create the parent directory, save a nonempty file, and close the figure.

Stage 3: covariance estimators and portfolios

`ridge_covariance(sigma, tau)` **Inputs:** covariance and nonnegative τ . **Work:** compute $\delta = \tau \text{tr}(\Sigma)/n$ and return $\Sigma + \delta I$. **Checks:** unchanged shape, symmetry, nondecreasing minimum eigenvalue, and an informative error for $\tau < 0$.

`equal_weight(n_assets)` **Input:** positive integer n . **Return:** $(n,)$ vector with entries $1/n$. **Checks:** finite nonnegative weights and budget error below tolerance.

`unconstrained_gmv(sigma)` **Input:** positive-definite covariance. **Work:** solve $\Sigma x = \mathbf{1}$ with `numpy.linalg.solve` and normalize $w = x/(\mathbf{1}^\top x)$; do not form an inverse. **Return:** $(n,)$ weights. **Checks:** nonzero normalization, budget feasibility, and stationarity $\Sigma w + \lambda \mathbf{1} \approx 0$ for the implied multiplier.

`solve_long_only_gmv(sigma, weight_cap, initial_weights)` **Inputs:** covariance, cap, and optional feasible start. **Work:** use SLSQP to minimize $\frac{1}{2}w^\top \Sigma w$ with $\mathbf{1}^\top w = 1$ and $0 \leq w_i \leq u$. **Return:** weights and raw `OptimizeResult`. **Reject:** $nu < 1$, invalid covariance, infeasible start, or failed solver. Here n is the number of assets and u the cap.

`solve_max_sharpe(mu, sigma, risk_free_rate, weight_cap, initial_weights)` **Inputs:** annual moments, annual risk-free rate, bounds, and start. **Work:** minimize negative Sharpe under the budget and bounds; evaluate several feasible initializations outside the function. **Return:** best successful weights and solver result. **Checks:** positive predicted variance, finite objective, feasibility, and no silent selection of a failed run.

`solve_mean_variance(mu, sigma, gamma, weight_cap)` **Inputs:** annual moments, $\gamma \geq 0$, and cap. **Work:** minimize $\frac{1}{2}w^\top \Sigma w - \gamma \mu^\top w$ with analytical gradient, budget, and bounds. **Return:** weights and solver result. **Checks:** units consistent, feasible cap, solver success, and constraints within tolerance.

`trace_efficient_frontier(mu, sigma, target_returns, weight_cap)` **Inputs:** annual moments, ordered target grid, and cap. **Work:** solve bounded minimum variance for every feasible target, warm-starting from the preceding solution. **Return:** one data-frame row per target containing requested and achieved return, volatility, concentration, weights or a weight key, solver status, message, and maximum violation. **Rule:** keep failed rows; do not hide them from the report.

`portfolio_diagnostics(weights, mu, sigma, lower, upper)` **Inputs:** one weight vector, annual moments, and bounds. **Return:** a dictionary containing budget error, lower and upper violations, predicted return, variance, volatility, concentration, minimum and maximum weights, and finite-value flag. **Checks:** compatible dimensions and variance not materially negative.

`directional_optimality_check(weights, objective, lower, upper, n_directions, step, seed)` **Inputs:** feasible weights, callable objective, bounds, and diagnostic settings. **Work:** generate reproducible budget-preserving directions, retain feasible perturbations, and evaluate objective changes. **Return:** number tested, smallest change, fraction improving, and seed. **Interpretation:** a detected improvement rejects numerical optimality; no detected improvement is not a proof.

Stage 4: model selection

`select_covariance_hyperparameters(train, candidate_k, candidate_tau, validation_length)` **Inputs:** training data only and candidate grids. **Work:** make a chronological estimation/validation split; refit every candidate on the estimation block; build GMV weights; evaluate realized validation volatility without re-estimating on validation. **Return:** one row per candidate with estimator name, hyperparameter, date boundaries, solver status, realized risk, concentration, and selection rank. **Checks:** no test-data argument, nonempty blocks, and deterministic tie-breaking.

Stage 5: backtesting

`drift_weights(post_trade_weights, realized_asset_return)` **Inputs:** two $(n,)$ arrays. **Work:** apply the self-financing drift formula after one realized return. **Return:** next pre-trade weights. **Checks:** positive portfolio gross return, finite output, budget preservation, and unchanged weights when all asset returns are equal.

`performance_metrics(net_returns, turnover, costs, risk_free_rate, annualization)` **Inputs:** aligned dated series and annual risk-free rate. **Work:** construct wealth and drawdown; compute compounded annual return, annual volatility, Sharpe, maximum drawdown, average rebalance turnover, total cost, and final wealth. **Return:** a scalar dictionary. **Reject:** empty, misaligned, duplicated-date, or nonfinite inputs.

`frozen_weight_backtest(test, weights, charge_initial_trade, cfg)` **Inputs:** test returns and one weight vector chosen from train. **Work:** apply constant target weights to every test date, follow the declared initial-cost convention, and compute all paths. **Return:** dictionary with gross/net returns, costs, wealth, drawdown, weights, and metrics. **Check:** weights never depend on test observations.

`backtest_rebalanced_strategy(train, test, covariance_estimator, optimizer, cfg, window)` **Inputs:** separated samples, two callables, configuration, and rolling or expanding. **Work:** at each date build an information set ending at $t - 1$, rebalance on schedule, compute turnover from pre-trade weights, apply post-trade weights to r_t , deduct costs, then drift. **Return:** aligned target and pre-trade weights, gross/net returns, turnover, costs, wealth, drawdown, solver logs, estimation-window dates, and summary metrics. **Checks:** one-period lag, feasible targets, budget after drift, and explicit handling of solver failure.

`plot_backtest_diagnostics(backtests, output_dir)` **Input:** dictionary of strategy backtest outputs. **Work:** align dates and create wealth, drawdown, rolling-volatility, turnover, and weight-heat-map figures with consistent colors. **Return:** None. **Checks:** labeled axes and units, legends, nonempty files, and closed figures.

Stage 6: orchestration and tests

`run_one_seed(seed)` **Input:** one integer seed. **Work:** run the same generation, selection, and backtest pipeline, reselecting hyperparameters inside that seed's training sample. **Return:** one tidy data frame with seed, strategy, selected parameters, and all metrics. **Rule:** never copy a choice made using another seed's test result.

`run_sanity_checks()` **Input:** none. **Work:** implement the deterministic checks listed in the skeleton, including identity-covariance GMV, PCA reconstruction, weight drift, invalid covariance rejection, and one-period lag. **Return:** None; raise immediately on failure and print one success message only after every check passes.

`main()` **Input:** none; read settings from CFG. **Work:** create directories, run sanity checks, generate or load data, fit diagnostics, select hyperparameters, construct portfolios, run backtests, save tables and plots, and execute multi-seed robustness. **Return:** None. **Requirement:** one top-to-bottom run from an empty data/output directory should reproduce every generated artifact.

4 Stage 0 – Set up a reproducible experiment

Use `seed=2026`, $n = 10$ assets, $k = 3$ factors, $T_{\text{train}} = 756$, and $T_{\text{test}} = 504$ trading days.

- 0.1. Create a configuration object containing all seeds, dimensions, annualization constants, transaction costs, and file paths. Avoid unexplained numbers inside functions.
- 0.2. Create separate random-number generators for population parameters, training returns, and test returns. Explain why separate streams make debugging easier.
- 0.3. Write a validation function that rejects non-finite arrays, unexpected dimensions, asymmetric covariance matrices, and materially negative eigenvalues.

- 0.4. Run the empty skeleton once and confirm that missing blocks fail with an explicit `NotImplementedError`, not a cryptic downstream error.

Self-check 0

Your report states the software versions, random seed, annualization convention, and directory structure. Running the project twice should produce identical data.

5 Stage 1 – Generate train and test returns

Generate returns from the factor model

$$r_t = \alpha + Bf_t + \varepsilon_t,$$

where

$$f_t \sim \mathcal{N}\left(\frac{\mu_f}{252}, \frac{\Sigma_f}{252}\right), \quad \varepsilon_t \sim \mathcal{N}\left(0, \frac{D}{252}\right).$$

Use annual factor means $\mu_f = (0.05, 0.025, 0.015)^\top$, annual factor volatilities $(0.16, 0.10, 0.07)$, and

$$R_f = \begin{pmatrix} 1 & 0.25 & -0.10 \\ 0.25 & 1 & 0.20 \\ -0.10 & 0.20 & 1 \end{pmatrix}, \quad \Sigma_f = \text{diag}(0.16, 0.10, 0.07) R_f \text{diag}(0.16, 0.10, 0.07).$$

Draw $B_{ij} \sim \mathcal{N}(0, 0.45^2)$, annual alphas uniformly between -1% and 3% , and annual idiosyncratic volatilities uniformly between 10% and 22% . If these volatilities are s_i , set $D = \text{diag}(s_1^2, \dots, s_n^2)$. Use the same population parameters for train and test, but independent shocks.

- 1.1. Construct Σ_f from volatilities and correlations. Check symmetry, diagonal entries, and smallest eigenvalue before simulating anything.
- 1.2. Generate α , B , and D . Record their shapes and units. Which quantities are annual and which are daily?
- 1.3. Derive and compute the theoretical annual moments

$$\mu_{\text{true}} = \alpha + B\mu_f, \quad \Sigma_{\text{true}} = B\Sigma_f B^\top + D.$$

- 1.4. Simulate train and test shocks independently. Verify that the same population parameters, not newly drawn parameters, are used for both samples.
- 1.5. Attach non-overlapping business-day indices. Name columns `Asset_01`, $\dots$, `Asset_10` and save exactly two return CSV files.
- 1.6. Save the population parameters in `true_parameters.npz`. Reload all three files and verify exact shapes, ordering, and absence of missing data.
- 1.7. Compare annualized sample means and covariances with theoretical values. Report relative Frobenius covariance error $\left\| \hat{\Sigma} - \Sigma_{\text{true}} \right\|_F / \left\| \Sigma_{\text{true}} \right\|_F$ for train and test.
- 1.8. Explain why the test estimates need not be closer to the truth even though the data-generating process is unchanged.

Self-check 1

Provide a table with sample sizes, missing-value counts, minimum covariance eigenvalue, mean-estimation error, and covariance-estimation error for each sample. Do not continue if units or dates are inconsistent.

6 Stage 2 – Learn covariance structure with PCA

Fit PCA only on centered training returns. You may use an eigendecomposition of the training covariance or an SVD of the centered data matrix. Never fit a transformation on the test sample.

- 2.1. Compute the training mean, center the training matrix, and verify that each centered column has mean numerically close to zero.

- 2.2. Compute eigenvalues and eigenvectors in decreasing order. Verify $U^T U \approx I$ and reconstruct the empirical covariance from all components.
- 2.3. Plot eigenvalues and cumulative explained variance. Mark $k = 3$. What feature of the data-generating process motivates three components?
- 2.4. Construct the rank-three and diagonal-corrected PCA covariances. Check symmetry, diagonal preservation, rank, and smallest eigenvalue.
- 2.5. Compare empirical and PCA covariances with $\Sigma_{\text{true}}/252$ using Frobenius error. Separate approximation error from estimation error in your discussion.
- 2.6. Compare empirical and theoretical leading subspaces. If s_j are singular values of $U_{\text{true}}^T U_{\text{PCA}}$, use principal angles $\theta_j = \arccos(\min(1, \max(-1, s_j)))$. Explain why eigenvector signs should not be compared directly.
- 2.7. Transform train and test returns using the training mean and loadings. Compare score variances without refitting PCA on test data.
- 2.8. Explain why PCA components need not recover the columns of B exactly, especially when idiosyncratic variances differ across assets.

Self-check 2

Suggested figures: scree plot, cumulative explained variance, empirical versus theoretical eigenvalues, and a heat map of the diagonal-corrected covariance. Suggested table: explained variance, principal angles, covariance errors, and condition numbers.

7 Stage 3 – Construct and validate portfolios

From training returns compute annualized $\hat{\mu}$ and $\hat{\Sigma}$. Build the following estimators: empirical covariance, diagonal-corrected PCA covariance for several k , and ridge covariance for a grid of τ values.

- 3.1. Implement a portfolio validator reporting budget error, minimum weight, maximum weight, predicted return, predicted volatility, concentration, and all stated constraint violations.
- 3.2. Implement equal weight and the closed-form unconstrained GMV portfolio. Check the closed-form stationarity equation before using an optimizer.
- 3.3. Solve long-only GMV with $0 \leq w_i \leq 0.30$. Report solver status and identify active lower and upper bounds using a numerical tolerance.
- 3.4. Solve long-only maximum Sharpe with $r_f = 2\%$. Guard against a zero-risk denominator and run several feasible initializations to detect local numerical issues.
- 3.5. Solve bounded mean–variance problems

$$\min_w \frac{1}{2} w^T \hat{\Sigma} w - \gamma \hat{\mu}^T w, \quad \mathbf{1}^T w = 1, \quad 0 \leq w_i \leq 0.30$$

for at least six logarithmically spaced values of γ .

- 3.6. Trace a target-return efficient frontier. Before solving, determine a feasible target interval under the weight bounds; do not ask the solver to reach an impossible return.
- 3.7. Compare empirical, PCA, and ridge covariances through eigenvalues, condition numbers, weights, concentration, predicted risk, and solver residuals.
- 3.8. Write down the KKT conditions for long-only GMV. If the solver exposes multipliers, compute stationarity and complementary-slackness residuals; otherwise verify primal feasibility and a directional optimality check.

Self-check 3

No portfolio enters the backtest unless its weights are finite, budget error is below 10^{-8} , bound violations are below 10^{-8} , predicted variance is nonnegative up to tolerance, and the solver reports success.

8 Stage 4 – Choose hyperparameters without test leakage

The number of PCA components k , ridge parameter τ , and mean–variance parameter γ are hyperparameters. Choose them using training information only.

- 4.1. Reserve the final 126 training observations as a chronological validation block. Estimate each candidate using only the earlier training observations.
- 4.2. State a selection criterion before examining validation results. A reasonable primary criterion is realized validation volatility, with average turnover or concentration as a tie-breaker.
- 4.3. Compare at least $k \in \{1, 2, 3, 5\}$ and $\tau \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. Explain why selecting the smallest in-sample predicted variance would favor overfitting.
- 4.4. Freeze the chosen rule before loading test returns. Store the selected parameters and criterion in a results table.
- 4.5. Optional: replace the single validation block with expanding or rolling time-series validation and compare the chosen hyperparameters.

9 Stage 5 – Backtest without look-ahead bias

Begin with a frozen-weight experiment: estimate once on the entire training sample, then hold the weights throughout the test sample. Next implement a rebalanced strategy.

At decision date t , use only observations strictly before t . Let w_t^- be pre-trade weights and w_t^+ post-trade weights. Between trades, weights drift according to

$$w_{i,t}^- = \frac{w_{i,t-1}^+(1 + r_{i,t-1})}{1 + (w_{t-1}^+)^T r_{t-1}}.$$

At a rebalance,

$$\text{TO}_t = \sum_i |w_{i,t}^+ - w_{i,t}^-|, \quad r_{p,t}^{\text{net}} = (w_t^+)^T r_t - c \text{TO}_t,$$

with $c = 0.001$. At a non-rebalance date set $w_t^+ = w_t^-$ and $\text{TO}_t = 0$.

- 5.1. Write a one-period-lag test: changing return r_t must not change the weights applied to r_t . Include this test in the code.
- 5.2. Rebalance every 21 trading days. Initially use the final 252 training observations; later use the most recent 252 observations available before the decision date.
- 5.3. Compare frozen weights, a 252-day rolling window, and an expanding window. State precisely which dates enter each estimate.
- 5.4. Before every trade, update drifting weights and verify they still sum to one. Then compute turnover relative to the new target weights.
- 5.5. Choose and report the initial-trade convention. Either begin from equal weight and charge turnover, or treat initial allocation as costless; use the same convention for every strategy.
- 5.6. Compute wealth $V_t = V_{t-1}(1 + r_{p,t}^{\text{net}})$ with $V_0 = 1$ and drawdown $1 - V_t / \max_{s \leq t} V_s$.
- 5.7. Report annualized return, volatility, Sharpe ratio, maximum drawdown, average turnover on trade dates, total transaction costs, and final wealth. Use daily risk-free return $r_{f,d} = (1 + r_f)^{1/252} - 1$.
- 5.8. Compare equal weight and empirical-, PCA-, and ridge-covariance GMV. Attribute differences separately to estimated risk, realized risk, concentration, turnover, and costs.

Self-check 5

Suggested diagnostics: wealth, drawdown, rolling 63-day volatility, turnover, weight heat maps, gross/net performance, and evidence that the first test weight excludes the first test return.

10 Stage 6 – Robustness and interpretation

- 6.1. Use the non-deployable oracle GMV portfolio from Σ_{true} only to quantify estimation error.
- 6.2. Repeat the experiment for at least five additional seeds, reselecting hyperparameters within each training sample. Plot the distributions of realized volatility, Sharpe ratio, drawdown, turnover, and final wealth; report medians and dispersion rather than means alone.
- 6.3. Across seeds, decide whether PCA or ridge consistently improves conditioning, weight stability, or realized risk. Discuss limitations from Gaussian returns, constant parameters, mean-estimation error, and simplified transaction costs.